

Civic.ai: A Bilingual Retrieval-Augmented Generation System for Bangladeshi Government-Service Question Answering

Merged Draft from Phase 1 and Phase 2

Prepared for thesis review and future integration into the official template

Current implementation domain: Birth and Death Registration

Date: June 2026

Abstract

Accessing government-service information in Bangladesh is often difficult because service procedures, fees, legal rules, correction instructions and frequently asked questions are scattered across different portals, documents and formats. Citizens may ask questions in Bangla, English or code-mixed language, while source documents are often noisy, Bangla-heavy and affected by formatting, OCR or encoding issues. A general chatbot is not sufficient for this setting because fluent answers without reliable evidence can create misinformation in public-service guidance.

This thesis proposes Civic.ai, a bilingual Retrieval-Augmented Generation (RAG) based question-answering system for Bangladeshi government services. The current phase focuses on the birth registration domain as the first testable implementation. The system uses a modular pipeline consisting of data collection, manual and semi-manual cleaning, document-type-aware chunking, metadata preservation, dense embedding, sparse retrieval, rank fusion, reranking, safe answer path checking and local LLM-based answer generation. To improve retrieval while preserving factual grounding, each chunk is represented using two separate text fields: `retrieval_text` for search and `answer_evidence` for generation.

The Phase 2 implementation evaluates multiple retrieval alternatives, including BM25-only retrieval, dense-only retrieval and hybrid RRF retrieval. A curated Bangla evaluation set with paraphrased citizen-style questions is used to measure retrieval behavior using Hit@k/Recall@k, Mean Reciprocal Rank (MRR), normalized Discounted Cumulative Gain (nDCG) and context precision. Preliminary results show that dense-only retrieval currently performs strongest for the single-domain birth registration dataset, while the full CivicRAG design remains important for explainability, source grounding and future multi-domain expansion. The system is implemented with local LLM backends such as llama3.2, llama3 and qwen2.5:7b so that the prototype can be tested without depending on external API keys.

The work contributes a practical and academically defensible design process for a Bangla-priority civic-service RAG system. It emphasizes factual correctness, retrieval quality, explainability, multilingual support and evaluation-driven design rather than general-purpose chatbot fluency.

Keywords: Retrieval-Augmented Generation, Civic AI, Bangla NLP, Government Services, Dense Retrieval, BM25, Reciprocal Rank Fusion, Local LLM

Contents

Abstract	1
1 Introduction	6
1.1 Introduction	6
1.2 Background	6
1.3 Rationale of the Study or Motivation	7
1.4 Problem Statement	7
1.5 Objectives	8
1.6 Methodology in Brief	8
1.7 Scope and Challenges	9
2 Literature Review	10
2.1 Preliminaries	10
2.2 Transformer and Large Language Models	10
2.3 Retrieval-Augmented Generation	11
2.4 Dense Retrieval, Sparse Retrieval, and Hybrid Retrieval	11
2.5 Multilingual and Bangla-Centric Considerations	11
2.6 AI in Government and Civic Services	12
2.7 Evaluation of RAG Systems	12
2.8 Research Gap	12
3 Proposed Methodology	13
3.1 Design Process or Methodology Overview	13
3.2 Preliminary Design Specification	14
3.2.1 Design Solutions	14
3.3 Data Collection	16

3.3.1	Data Cleaning	16
3.3.2	Data Transformation	17
3.3.3	Data Integration	18
3.3.4	Data Reduction	18
3.3.5	Summary of Preprocessed Data	19
3.4	Implementation of Selected Design	19
3.5	Preliminary Evaluation	20

List of Figures

- 3.1 Proposed CivicRAG architecture for the birth registration domain. The offline indexing pipeline is separated from the online question-answering pipeline so that retrieval quality and generation quality can be evaluated independently. 14
- 3.2 Data cleaning and preprocessing pipeline for proposed CivicRAG. 17

List of Tables

3.1	Design solutions considered for Civic.ai.	15
3.2	Chunking strategy by document type.	17
3.3	Retrieval variant comparison on the 58-question Bangla evaluation set. . .	20
3.4	Generation context top-k coverage.	21

Chapter 1

Introduction

1.1 Introduction

This chapter presents the background, motivation, problem statement, objectives, scope, and methodological direction of the thesis. The project, Civic.ai, is designed as a bilingual and evidence-grounded government-service question-answering system for Bangladesh. The work began with a broad idea of an AI-powered civic assistant, but the current thesis direction is more specific: building and evaluating a Retrieval-Augmented Generation (RAG) based chatbot that retrieves trusted government-service evidence before producing an answer.

The main focus of the current phase is the birth and death registration domain. This narrow scope is intentional. Government-service information is sensitive because inaccurate guidance can waste time, money, and trust. Therefore, before expanding to future domains such as passport, BRTA, birth certificate, death certificate, or other public services, the thesis first studies whether a carefully designed RAG system can provide grounded and explainable answers for one domain.

1.2 Background

Digital government services are now an important part of public administration. Bangladesh has introduced many online portals and digital services to make public service delivery more accessible. However, citizens still face practical difficulties while using these systems. Information is often scattered across multiple websites, service portals, notices, PDF files, FAQ pages, and legal documents. Many documents contain Bangla, English, mixed-language terms, tables, scanned pages, or formatting noise. As a result, even when information is technically available online, it is not always easy for ordinary citizens to find, understand, and apply.

Recent progress in natural language processing has made conversational systems more capable. The Transformer architecture introduced self-attention as a scalable method for language modeling [8]. Large language models such as LLaMA show that open-weight models can be used for local language generation tasks [23], [26]. Bangla-focused models

and resources, such as BanglaBERT, show the importance of language-specific modeling for low-resource languages like Bangla [18]. However, general-purpose LLMs alone are not enough for government-service question answering because they may hallucinate or produce unsupported claims.

Retrieval-Augmented Generation addresses this limitation by connecting a language model with an external knowledge source. In a RAG system, the retriever first finds relevant evidence from a document collection, and the generator then uses that evidence to produce an answer [16]. This design is suitable for Civic.ai because factual correctness, source grounding, and explainability are more important than creative fluency.

1.3 Rationale of the Study or Motivation

The motivation for this research comes from both social need and technical opportunity. From the citizen’s perspective, many government-service procedures remain difficult to understand. People often depend on unofficial intermediaries to fill forms, understand requirements, or know where to go next. This can increase cost, delay, confusion, and risk of misinformation.

From the technical perspective, RAG provides a practical way to build a trustworthy assistant without fine-tuning a large model on every government document. Instead of depending only on the model’s internal memory, Civic.ai retrieves evidence from curated government-service documents. This allows the system to be updated when documents change, makes answers easier to audit, and supports source-based verification.

The academic motivation is to study how a bilingual Bangla-English RAG pipeline can be designed for noisy real-world government documents. Unlike clean tutorial datasets, government-service data includes PDFs, converted DOCX files, HTML pages, FAQs, tables, notices, legal rules, scanned documents, encoding problems, and repeated web boilerplate. Therefore, this thesis emphasizes data cleaning, document-type-aware chunking, metadata handling, retrieval evaluation, and grounded answer generation.

1.4 Problem Statement

Despite the growth of e-government services, citizens in Bangladesh still face several barriers:

- **Dispersed information:** Service instructions, document requirements, fees, and correction procedures are spread across different pages and document formats.
- **Language and wording barriers:** Citizens may ask in Bangla, English, or code-mixed language, while source documents may use formal Bangla, English terms, or official phrases.
- **Document noise:** Government pages often include menus, footers, accessibility text, duplicated links, scanned content, and formatting artifacts that can damage retrieval quality.

- **Lack of interactive guidance:** Existing portals mostly provide static pages or FAQs, not scenario-based conversational support.
- **Risk of hallucination:** A general chatbot may produce fluent but unsupported answers, which is unsafe for public-service guidance.

Research Problem: How can a bilingual, retrieval-based AI assistant be designed and evaluated so that it gives accurate, source-grounded, and explainable answers for Bangladeshi government-service questions?

1.5 Objectives

Overall Objective: To design, implement, and evaluate Civic.ai, a bilingual RAG-based government-service question-answering system that prioritizes factual correctness, retrieval quality, and explainability.

Specific Objectives:

- Build a modular RAG pipeline with clear separation between ingestion, cleaning, chunking, embedding, retrieval, reranking, generation, and evaluation.
- Prepare a Bangla-heavy government-service knowledge base from noisy real-world sources.
- Design document-type-aware chunking for FAQ, rule, fee-table, application-process, and correction-process documents.
- Evaluate dense retrieval, BM25 retrieval, and hybrid retrieval alternatives using retrieval metrics such as Hit@k/Recall@k, MRR, nDCG, and context precision.
- Compare local LLM backends such as llama3.2, llama3, and qwen2.5:7b for grounded answer generation.
- Produce answers with source IDs, retrieval scores, and official links so that users and evaluators can inspect the evidence.

1.6 Methodology in Brief

The thesis follows an iterative design and evaluation methodology. First, raw government-service documents are collected from official and related sources. Second, the documents are manually and semi-manually cleaned to remove noise, duplicate navigation text, encoding issues, and irrelevant fragments. Third, cleaned files are converted into typed chunks according to document structure. Fourth, each chunk is stored with metadata and two text versions: a search-optimized `retrieval_text` and an original `answer_evidence` field for grounding.

The retrieval pipeline uses multilingual dense embeddings through BAAI/bge-m3 [25], a sparse BM25 index [5], and rank fusion through Reciprocal Rank Fusion [4]. The answer

generation layer uses local Ollama models so that the system can run without external API keys. Evaluation is performed in two parts: retrieval-side evaluation using a curated Bangla question set, and generation-side evaluation through faithfulness, relevance, correctness, stability, and manual qualitative review.

1.7 Scope and Challenges

The current implementation scope is limited to the birth and death registration domain. This includes application procedures, correction guidance, fees, FAQ entries, legal rules, and general service guidance. The system architecture is still designed as a multi-domain system so that future domains can be added under separate domain folders without creating separate repositories.

The main challenges include noisy source data, Bangla and mixed-language retrieval, variation between citizen wording and official wording, local hardware limitations, hallucination risk, and the need for reliable evaluation data. These challenges guide the design choices made in Chapter 3.

Chapter 2

Literature Review

2.1 Preliminaries

The technical foundation of this thesis comes from recent progress in transformer-based language modeling, multilingual NLP, retrieval systems, and RAG-based question answering. The Transformer architecture replaced recurrent sequence processing with self-attention, enabling scalable representation learning for natural language [8]. Modern large language models extend this idea by training on large text corpora and generating fluent responses to user prompts [23], [26].

For Bangla and other low-resource languages, language-specific and multilingual representation learning is important. BanglaBERT demonstrates that pretraining on Bangla corpora can improve downstream Bangla NLP performance [18]. However, Civic.ai does not currently fine-tune a Bangla language model. Instead, it uses multilingual retrieval and local LLM generation because the main thesis goal is evidence-grounded government-service QA, not training a new foundation model.

2.2 Transformer and Large Language Models

Transformer-based LLMs can understand and generate natural language, but their knowledge is stored parametrically and may become outdated or unsupported. In public-service guidance, this is risky because users may rely on the answer to make decisions about documents, fees, correction procedures, or office visits. LLMs can also hallucinate, meaning they may produce statements that sound plausible but are not grounded in evidence. Verification-oriented work such as Chain-of-Verification shows that explicit checking can reduce hallucination in LLM outputs [20]. For Civic.ai, this supports the decision to use retrieval grounding and controlled answer paths for high-risk civic questions.

2.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation combines two components: a retriever and a generator. The retriever searches an external knowledge base, while the generator produces an answer conditioned on retrieved passages [16]. RAG is especially useful for knowledge-intensive tasks because source documents can be updated without retraining the language model.

Recent RAG surveys describe multiple stages of RAG evolution, including naive RAG, advanced RAG, modular RAG, hybrid retrieval, reranking, query transformation, and evaluation [21], [28]. These studies show that retrieval quality is often the most important factor in answer correctness. If the retriever returns irrelevant or noisy chunks, even a strong LLM may generate a poor answer. This directly supports the thesis priority: accuracy first, retrieval quality second, reasoning third, and fluency last.

A related bilingual Bangla-English RAG study, NextRAG, shows the relevance of pre-processing, hybrid retrieval, and local LLM comparison in a Bangladeshi QA context [31]. Civic.ai is inspired by the general direction of such work, but it applies the design to government-service guidance instead of financial question answering.

2.4 Dense Retrieval, Sparse Retrieval, and Hybrid Retrieval

Dense retrieval represents queries and documents as vectors and retrieves content based on semantic similarity. Dense Passage Retrieval showed that learned dense representations can improve open-domain question answering [15]. Sentence-BERT also demonstrated that sentence-level semantic embeddings are useful for similarity and retrieval tasks [12]. In Civic.ai, dense retrieval is important because citizen questions are often paraphrased. For example, a user may ask about a lost birth certificate in informal wording that does not exactly match the official FAQ title.

Sparse retrieval, such as BM25, ranks documents based on lexical term matching and term weighting [5]. BM25 remains useful for exact government-service terms, fee names, legal references, and official terminology. Hybrid retrieval combines dense and sparse signals to reduce the weaknesses of either method alone. Reciprocal Rank Fusion is a rank-level fusion method that combines ranked lists without requiring raw dense and sparse scores to be directly comparable [4].

2.5 Multilingual and Bangla-Centric Considerations

A Bangladeshi government-service assistant cannot be designed as an English-only system. Users may ask questions in Bangla, English, or code-mixed Bangla-English. Source documents may contain formal Bangla, English service terms, legal expressions, and OCR or encoding issues. This makes multilingual embeddings important. BGE-M3 is selected because it is designed for multilingual, multi-functionality, and multi-granularity retrieval [25]. This supports cross-lingual and paraphrase-based retrieval better than an English-only embedding model.

The system also separates `retrieval_text` from `answer_evidence`. Retrieval text may include aliases and normalized wording to improve search. Answer evidence preserves original Bangla source content so that the LLM is grounded on authentic evidence rather than artificial query-expansion text. This distinction is central to making the system both searchable and faithful.

2.6 AI in Government and Civic Services

Research on AI in government highlights opportunities for efficiency, transparency, and citizen engagement, but it also raises concerns about trust, accountability, and data governance [19], [24], [30]. Public-service chatbots must be more careful than casual assistants because wrong answers may directly affect citizens’ decisions. Studies on chatbots in public service also show that user trust depends on transparency, reliability, and clear expectation setting [7], [22], [29].

For this reason, Civic.ai is designed to show source IDs, retrieval scores, and official/source links. The goal is not only to answer the user but also to make the answer inspectable.

2.7 Evaluation of RAG Systems

Evaluation is essential for thesis-level RAG work. General NLP benchmarks such as GLUE and SuperGLUE evaluate language understanding, while XQuAD and TyDi QA evaluate multilingual and cross-lingual question answering [9], [10], [13], [14]. However, a domain-specific RAG system also needs retrieval-specific and generation-specific evaluation.

Information retrieval metrics such as $\text{Hit}@k/\text{Recall}@k$, Mean Reciprocal Rank, and nDCG are used to measure whether expected evidence appears in the retrieved results and how highly it is ranked [1]–[3]. RAGAS proposes automated metrics such as faithfulness, answer relevance, and context relevance for retrieval-augmented systems [27]. When reference answers are available, QA-style exact match or F1 can be used [6]. When wording differs but meaning is similar, semantic metrics such as BERTScore are useful [17].

2.8 Research Gap

Existing work shows strong progress in LLMs, RAG, multilingual NLP, and AI for public service. However, there is still a practical gap for Bangla-English government-service RAG systems built from noisy Bangladeshi documents. Many studies focus on clean datasets, English-heavy benchmarks, or broad chatbot behavior. Civic.ai addresses this gap by focusing on Bangla-heavy government-service documents, document-type-aware chunking, retrieval/generation separation, local LLMs, and measurable evaluation of retrieval alternatives.

Chapter 3

Proposed Methodology

3.1 Design Process or Methodology Overview

In this phase, we focused on transforming the high-level idea of Civic.ai into a concrete, testable system design. Previously, the work was framed as a citizen-centric AI-powered government service assistant. But now, the design has been refined into an evidence-based Retrieval-Augmented Generation (RAG) system for Bangladeshi government service question answering. We have limited the current implementation scope to the birth registration domain only so that the design can be evaluated carefully before adding additional domains in future such as passport, BRTA or other civic services.

The central design requirement of our system is factual correctness. We expect that a government-service assistant should not respond like a general creative chatbot. Rather, it should retrieve relevant evidence from the trusted documents, generate answers from that evidence, give source information and restrict any unsupported claim. For this reason, RAG based design is used in our Civic.ai system, where a retriever first selects relevant document chunks and a generator then produces an answer grounded in the retrieved evidence [16].

The design process follows five major stages:

1. Collection of raw government-service documents from birth registration sources.
2. Manual and semi-manual cleaning to remove web noise, duplicate navigation text, OCR artifacts, and formatting inconsistencies.
3. Document aware chunking for different types such as FAQ, rule, fee-table, correction process and application process documents.
4. Indexing with both dense multilingual embeddings and sparse lexical retrieval.
5. Retrieval, fusion, reranking, answer generation and evaluation using curated Bangla scenario-based questions.

Figure 3.1 shows the overall architecture of our current system. It is divided into two pipelines. The offline pipeline at first, processes raw government documents, cleans and

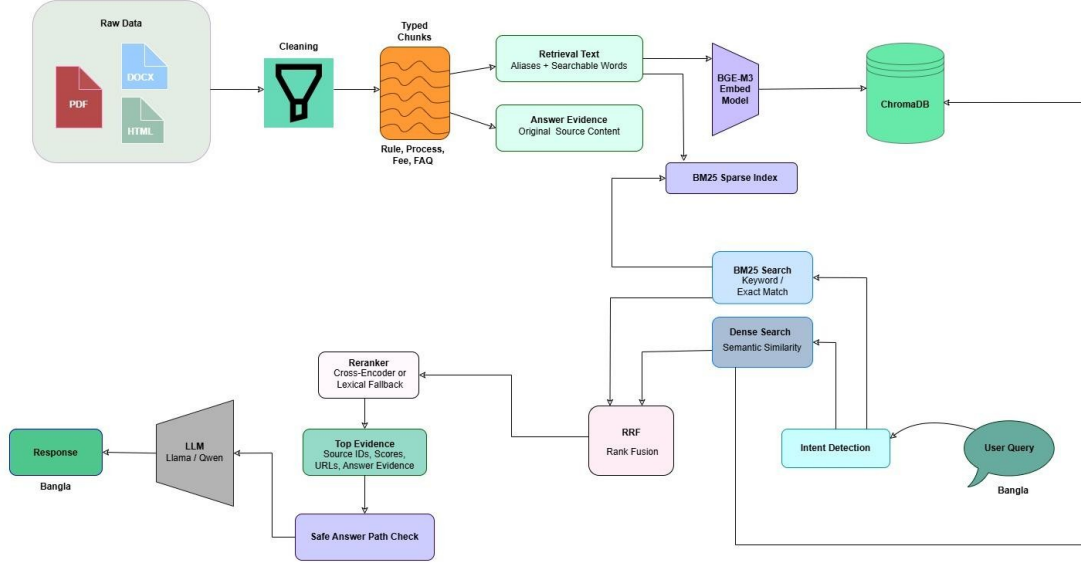


Figure 3.1: Proposed CivicRAG architecture for the birth registration domain. The offline indexing pipeline is separated from the online question-answering pipeline so that retrieval quality and generation quality can be evaluated independently.

chunks the data. Then, it creates two text versions per chunk, embeds the searchable text and stores it in a vector database. The online pipeline accepts a user query, detects language and civic intent, retrieves candidate chunks using dense and sparse retrieval. After that, it fuses ranked results, reranks evidence and finally generates a grounded response using either controlled answer paths or a local LLM.

3.2 Preliminary Design Specification

We considered and simulated several design solutions. The purpose of this comparison was not just to choose one final system but, additionally to justify why the selected design is appropriate for a factual Bangla government-service assistant.

3.2.1 Design Solutions

The dense retrieval component is motivated by prior work on dense passage retrieval for open domain question answering [15] and sentence-level semantic representation learning [12]. In our current implementation, we used the BAAI/bge-m3 embedding model as it is designed for multilingual, multi-functionality and multi-granularity retrieval [25]. This is important for our Civic.ai system because our knowledge base is Bangla-heavy while user queries may appear in Bangla, English or code-mixed form.

The sparse retrieval component uses BM25, a strong probabilistic lexical retrieval baseline [5]. BM25 is specifically useful when a query contains exact service words, fee-related words, rule references, or official terminology. The simplified BM25 scoring function can be represented as:

Table 3.1: Design solutions considered for Civic.ai.

Design Alternative	Description	Reason for Evaluation
Dense-only RAG	Uses multilingual dense embeddings and vector similarity search.	Strong candidate for paraphrased Bangla queries where exact words may differ from source documents.
BM25-only RAG	Uses sparse lexical retrieval based on term matching.	Useful for exact government terms, legal rule numbers, fee terms and service names [5].
Hybrid RAG	Combines dense retrieval and BM25 using rank fusion.	Provides a more robust architecture for future multi-domain expansion where both semantic and exact matching may matter.
CivicRAG	Adds metadata, source URLs, language and intent detection, reranking, safe answer paths and local LLM generation.	Selected design because it supports explainability, factual grounding and controlled answers for high-risk civic service questions.

$$BM25(q, d) = \sum_{t \in q} IDF(t) \cdot \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1(1 - b + b \cdot \frac{|d|}{avgdl})} \quad (3.1)$$

Here, q is the query, d is a document chunk, $f(t, d)$ is the frequency of term t in chunk d , $|d|$ is the chunk length and $avgdl$ is the average chunk length. The parameter k_1 controls term-frequency and b controls saturation and length normalization [5].

To combine dense and sparse retrieval results, the system uses Reciprocal Rank Fusion (RRF). RRF combines multiple ranked lists without requiring the raw scores of different retrievers to be directly comparable [4]. In the current system, a weighted form of RRF is used:

$$RRF(d) = \sum_{r \in R} \frac{w_r}{k + rank_r(d)} \quad (3.2)$$

where d is a candidate chunk, R is the set of retrievers, w_r is the retriever weight, k is the RRF constant, and $rank_r(d)$ is the rank of the chunk d under retriever r . The original RRF formulation is unweighted, but the weighted version is useful for Phase 2 sensitivity analysis because it allows the dense and BM25 contributions to be tuned.

After going through fusion, a reranking stage is used to improve the order of candidate chunks. Cross encoder reranking is a common approach for passage reranking because it jointly scores the query and candidate passage [11]. The current Civic.ai implementation supports cross-encoder reranking when the local model is available and uses deterministic lexical/domain-aware fallback reranking for local reproducibility.

3.3 Data Collection

The current dataset focuses on Bangladeshi birth registration services. The sources include government service web pages, frequently asked questions, legal rules, fee information, application process documents, correction guidance and scanned or converted documents related to birth registration.

Four major raw data types were considered:

- Bijoy-encoded PDF files.
- Converted DOCX files.
- Unicode HTML pages from government web portals.
- Scanned PDF or OCR-affected documents.

The current Phase 2 implementation primarily indexes manually reviewed and cleaned Markdown/JSON data. Raw scanned and converted files are retained for auditability, but only cleaned and structured content is used for the active vector database. This decision was made to reduce garbage-in-garbage-out errors in retrieval.

3.3.1 Data Cleaning

Civic service documents are often noisy and mixed or diverse. HTML files often contain menus, accessibility controls, footers, lists of unrelated services and navigation text. PDF and DOCX files sometimes contain broken formatting, repeated headings, encoding issues and OCR noise. Sometimes they include diagrams as well. Data cleaning is considered and treated as one of the core design steps rather than something minor since the quality of RAG heavily depends on the quality of retrieved data.

The data cleaning process includes:

- Removing menus, footer blocks, accessibility controls and unrelated service lists.
- Removing gibberish texts and symbols and irrelevant text.
- Removing and converting all the non-Unicode/Bijoy texts where possible.
- Turning FAQ entries into individual question-answer units for creating safe answering paths.
- Preserving legal and procedural wording where exact phrasing is important.
- Eliminating OCR fragments, empty headings and duplicate content.
- Preserving official source URLs as metadata for explainability.

The goal of cleaning data is to prevent irrelevant and unnecessary texts from being embedded and retrieved. If noisy navigation or meaningless text is stored in the vector database the retriever may return chunks that match common words but do not answer the citizen's question or include gibberish texts in the response.

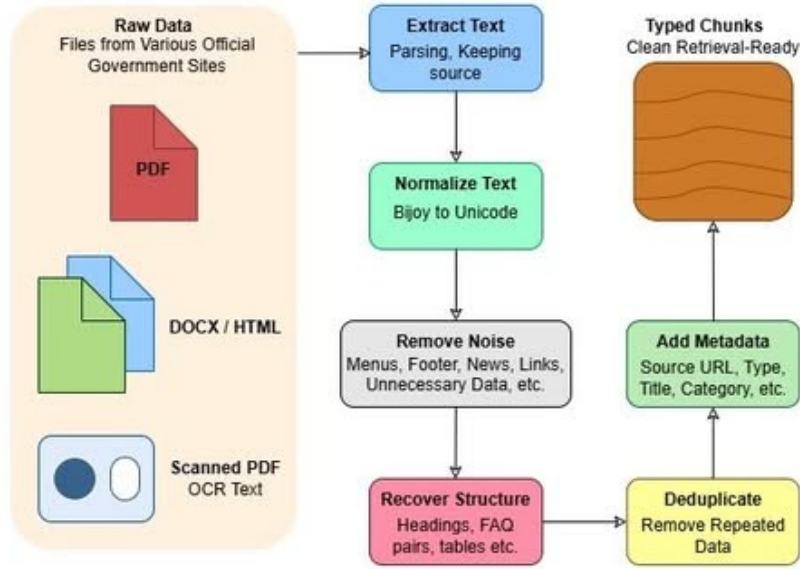


Figure 3.2: Data cleaning and preprocessing pipeline for proposed CivicRAG.

3.3.2 Data Transformation

The data is transformed into retrieval friendly useful chunks after the cleaning process. Since there are various kinds of document types and one single chunking method is not suitable for that, we are using document-type-aware chunking.

Table 3.2: Chunking strategy by document type.

Document Type	Chunking Strategy	Reason
FAQ JSON	Question-answer pair wise JSON chunk.	FAQ answers the most accurate answers and it is better to keep them intact.
Heading based Markdown	Section and subsection based chunking.	Preserves procedural structure and improves evidence interpretability.
Fee tables	Row wise chunks with fee metadata.	Fee questions require precise row level retrieval.
Legal/rule documents	Rule or paragraph group based chunks.	Legal evidence should be separated from unrelated sections.
Correction/application process documents	Step or condition based chunks.	Queries often include specific scenarios or require procedural steps.

Each chunk contains two separate text fields:

- **retrieval_text**: used for searching. It may include aliases, normalized terms and key words.

- **answer_evidence:** used for generation. It contains original source content and is passed to the LLM for grounding.

This separation is required because aliases can elevate retrieval quality but the generator should not be grounded on artificial aliases. Therefore, the LLM receives original source content rather than expanded retrieval text.

3.3.3 Data Integration

All domain specific resources are organized under one domain. The current domain is:

```
domains/birth_death_registration/
```

This folder holds cleaned Markdown/JSON files, generated chunks, ChromaDB vector storage, configuration files and evaluation datasets including raw documents. This design will help future multi domain expansion without having to create separate repositories. Future domains such as passport or BRTA as discussed can be added as new domain folders with their own data and configuration files.

3.3.4 Data Reduction

In this project, data reduction does not actually mean numerical dimensionality reduction. Instead, it means reducing irrelevant document noise and gibberish texts before indexing and reducing retrieved candidate chunks before generation. The important reduction decisions are:

- Eliminating duplicate and irrelevant government website boilerplate.
- Removing duplicate FAQ content where the same question/answer appears in multiple sources.
- Splitting long documents into focused evidence units.
- Using fine tuned top-k retrieval to avoid sending too much context to the generator.
- Applying reranking and intent based boosting to prioritize the most relevant evidence instead of blindly sending the retrieved evidence.

Data Reduction is necessary because long or noisy retrieved data can confuse LLM generation. In a government service system, the answer should be precise and provide enough information but controlled because the actual answer should not be influenced or overshadowed by the unrelated chunks.

3.3.5 Summary of Preprocessed Data

The final preprocessed data contains Bangla heavy birth registration chunks. The major chunk categories are FAQ, legal rules, fee rows, correction steps, application steps, portal guidance and general service information. Each chunk holds metadata such as source ID, document type, section title, category and source URL.

For Phase 2 evaluation, a 58-question Bangla paraphrase evaluation set was created from 19 grouped birth certificate related problems. These questions are designed to evaluate whether the system can retrieve the same expected evidence when a citizen asks the same problem in different wording and even if it does how well it can do it.

3.4 Implementation of Selected Design

The selected design is CivicRAG, a modular local RAG system for government service question answering. The implementation consists of six main modules:

1. **Data processing module:** reads cleaned Markdown and JSON files and converts them into typed chunks.
2. **Embedding module:** embeds `retrieval_text` using BAAI/bge-m3 [25].
3. **Indexing module:** stores dense vectors in ChromaDB and builds a BM25 sparse index.
4. **Retrieval module:** retrieves candidates using dense search, BM25 search, or hybrid RRF search.
5. **Reranking and safety module:** reranks candidates and checks whether a controlled answer path is available.
6. **Generation module:** produces a final answer using local LLMs when a controlled answer is not available.

The current local LLM backends are llama3.2, llama3, and qwen2.5:7b. Llama-based models are used as local open-weight baselines [26], while Qwen2.5 is included as an alternative multilingual local model [32]. The default generation temperature is set to 0.05 since government service queries require precise and consistent answers rather than creative answers. This low temperature eliminates the randomness.

For high-confidence civic intents such as fee questions, lost certificate questions, correction questions and application-process based questions the system can use safe answer paths. These paths reduce unnecessary LLM intervention by producing answers directly from selected evidence patterns. Retrieved evidence is sent to the local LLM with a strict grounding prompt to generate an answer if no safe path is found.

Generation quality is decided to be evaluated using metrics such as answer correctness, answer relevance, faithfulness and qualitative human review. RAG-specific evaluation

frameworks such as RAGAS emphasize faithfulness, answer relevancy and context quality for retrieval augmented systems [27].

Therefore, the Phase 2 implementation satisfies the design requirement of evaluating multiple alternative solutions before selecting the final architecture. Dense-only retrieval currently has the highest score on all fields, therefore, is the strongest retrieval baseline on the single domain Bangla evaluation set. Although CivicRAG remains the chosen system design since it is more explainable, supports source grounded responses and is better prepared for future multi-domain civic service expansion.

3.5 Preliminary Evaluation

A curated Bangla evaluation set was created to simulate real citizen questions. The current evaluation set contains 58 paraphrased questions. These questions cover application processes, correction cases, lost certificate cases, fees, legal conditions, online visibility problems and document requirements.

The retrieval alternatives were evaluated using ranking metrics commonly used in information retrieval, including Hit@k/Recall@k, Mean Reciprocal Rank (MRR), normalized Discounted Cumulative Gain (nDCG) and Context Precision [1]–[3]. The main retrieval results are shown in Table 3.3.

Table 3.3: Retrieval variant comparison on the 58-question Bangla evaluation set.

Method	Hit@1	Hit@5	MRR	nDCG@5
BM25-only	0.5345	0.7586	0.6132	0.6112
Dense-only	0.7931	0.9828	0.8707	0.8693
Hybrid RRF	0.7414	0.9483	0.8193	0.8139

The result shows that dense-only retrieval currently performs best for the birth registration evaluation set. This is reasonable because many evaluation questions are slightly modified or paraphrased Bangla questions, where semantic similarity is more important than exact word overlap. However, hybrid retrieval remains part of the selected design because future multi-domain expansion may increase lexical ambiguity. For example, different government services may contain semantically similar instructions but domain-specific terms such as passport, driving license, birth registration, death registration, fee, correction, or application. In such settings, sparse retrieval can help preserve exact domain signals.

The following metrics were used:

$$Recall@k = \frac{\text{Number of queries where a relevant chunk appears in top } k}{\text{Total number of queries}} \quad (3.3)$$

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i} \quad (3.4)$$

MRR measures how early the first relevant chunk appears in the ranked list [1], [3]. For ranking quality with graded relevance, nDCG is used:

$$DCG@k = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i + 1)} \quad (3.5)$$

$$nDCG@k = \frac{DCG@k}{IDCG@k} \quad (3.6)$$

where rel_i is the relevance score of the result at rank i , and $IDCG@k$ is the ideal DCG at rank k [2].

The generation side design was also examined through context size testing. The goal was to determine how many retrieved chunks should be sent to the answer generator. Table 3.4 shows the trade-off between evidence coverage and context precision.

Table 3.4: Generation context top-k coverage.

k	Expected Source in Context	Context Precision@k
1	0.7414	0.7414
3	0.8793	0.3621
5	0.9483	0.2448
8	0.9828	0.1616

The results show that larger context sizes improve the chance of including the expected evidence but also introduce more irrelevant chunks. The current selected value is $k = 3$ for generation because it is conservative and reduces noise. However, $k = 5$ is a candidate for future testing because it improves expected-source coverage.

Bibliography

- [1] E. M. Voorhees and D. M. Tice, “The trec-8 question answering track report,” in *Proceedings of the Eighth Text REtrieval Conference*, 1999.
- [2] K. Jarvelin and J. Kekalainen, “Cumulated gain-based evaluation of ir techniques,” *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002.
- [3] C. D. Manning, P. Raghavan, and H. Schutze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [4] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.
- [5] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: Bm25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [6] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “Squad: 100,000+ questions for machine comprehension of text,” in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2016, pp. 2383–2392.
- [7] H. Mehr, “Artificial intelligence for citizen services and government,” *Ash Center for Democratic Governance and Innovation, Harvard Kennedy School*, 2017.
- [8] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017.
- [9] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, “Glue: A multi-task benchmark and analysis platform for natural language understanding,” *arXiv preprint arXiv:1804.07461*, 2018.
- [10] M. Artetxe, S. Ruder, and D. Yogatama, “On the cross-lingual transferability of monolingual representations,” *arXiv preprint arXiv:1910.11856*, 2019.
- [11] R. Nogueira and K. Cho, “Passage re-ranking with bert,” *arXiv preprint arXiv:1901.04085*, 2019.
- [12] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992.
- [13] A. Wang, Y. Pruksachatkun, N. Nangia, *et al.*, “Superglue: A stickier benchmark for general-purpose language understanding systems,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.

- [14] J. H. Clark, E. Choi, M. Collins, *et al.*, “Tydi qa: A benchmark for information-seeking question answering in typologically diverse languages,” *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 454–470, 2020.
- [15] V. Karpukhin, B. Oguz, S. Min, *et al.*, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020, pp. 6769–6781.
- [16] P. Lewis, E. Perez, A. Piktus, *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, 2020.
- [17] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “Bertscore: Evaluating text generation with bert,” in *International Conference on Learning Representations*, 2020.
- [18] A. Bhattacharjee, T. Hasan, W. U. Ahmad, *et al.*, “Banglabert: Language model pretraining and benchmarks for low-resource language understanding evaluation in bangla,” *arXiv preprint arXiv:2101.00204*, 2021.
- [19] P. Andrews, T. de Sousa, B. Haefele, *et al.*, “A trust framework for government use of artificial intelligence and automated decision making,” *arXiv preprint arXiv:2208.10087*, 2022.
- [20] S. Dhuliawala, M. Komeili, J. Xu, *et al.*, “Chain-of-verification reduces hallucination in large language models,” *arXiv preprint arXiv:2309.11495*, 2023.
- [21] Y. Gao, Y. Xiong, X. Gao, *et al.*, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2023.
- [22] Z. Lin, “How generative-ai can be effectively used in government chatbots,” *arXiv preprint arXiv:2312.02181*, 2023.
- [23] H. Touvron, T. Lavril, G. Izacard, *et al.*, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [24] K. Alhosani and S. M. Alhashmi, “Opportunities, challenges, and benefits of ai innovation in government services: A review,” *Discover Artificial Intelligence*, vol. 4, no. 1, p. 18, 2024.
- [25] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, *Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation*, 2024. arXiv: 2402.03216 [cs.CL].
- [26] A. Dubey *et al.*, *The llama 3 herd of models*, 2024. arXiv: 2407.21783 [cs.AI].
- [27] S. Es, J. James, L. E. Anke, and S. Schockaert, “Ragas: Automated evaluation of retrieval augmented generation,” in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 2024, pp. 150–158.
- [28] S. Gupta, R. Ranjan, and S. N. Singh, “A comprehensive survey of retrieval-augmented generation: Evolution, current landscape and future directions,” *arXiv preprint arXiv:2410.12837*, 2024.
- [29] A. G. Larsen and A. Folstad, “The impact of chatbots on public service provision: A qualitative interview study with citizens and public service providers,” *Government Information Quarterly*, vol. 41, no. 2, p. 101927, 2024.

- [30] M. Pislaru, C. S. Vlad, L. Ivascu, and I. I. Mircea, “Citizen-centric governance: Enhancing citizen engagement through artificial intelligence tools,” *Sustainability*, vol. 16, no. 7, p. 2686, 2024.
- [31] S. A. I. Opin, R. A. Thakur, A. K. Raiyan, M. R. Kabir, and S. Rahman, “Nex-trag: A bilingual retrieval-augmented generation framework for financial question-answering,” in *2025 28th International Conference on Computer and Information Technology (ICCIT)*, IEEE, 2025. DOI: 10.1109/ICCIT68739.2025.11491522.
- [32] Qwen Team, *Qwen2.5 technical report*, Technical report, 2025.